

# Beyond Point Dates: Exact Evidence Selection for Temporal Retrieval

Anonymous ACL submission

## Abstract

Sources often date an update only to a month or year. Historical questions can also cover entire periods. Assigning exact dates can then hide uncertainty and discard useful evidence. We introduce a temporal retrieval compiler that preserves the source’s date constraints. It stores at most two replacement records for each claim. These records distinguish evidence that could apply from evidence that must apply somewhere within the requested period. We prove that two records suffice and can be necessary. We also identify exactly when date uncertainty cannot change the ranked claim context. Each mistaken directed replacement decision affects only its target claim. Independent oracles match all 253,050 tested support decisions and 328,747 context-stability decisions. Experiments preserve complete TempLAMA answer sets and original TimeQA paragraphs. A latest-value baseline suppresses 13.63% of accepted annual answers by keeping only one value. The results establish an exact selection interface and a source-grounded protocol for temporal evidence.

## 1 Introduction

A useful historical answer needs the right period. Consider a question about a radio programme’s hosts between 1977 and 1995. Several successive hosts belong in the answer. Keeping only the latest host loses part of that history. Likewise, a stadium can host two teams during overlapping periods. A newer team does not automatically replace an older tenant. Both examples occur in the natural passages of TimeQA (Chen et al., 2021).

Retrieval-augmented generation must fit this evidence into a limited context (Lewis et al., 2020). Text relevance identifies the topic. Temporal selection determines which descriptions can support the requested period. That selection becomes harder when sources give coarse dates. An appointment reported as occurring in a year does not identify its

exact day. A later record can establish a possible replacement without establishing when it happened. Committing to an arbitrary date hides this distinction from the reader.

Existing temporal memories already preserve fact histories and invalidate earlier claims. Graphiti applies direct contradiction decisions (Rasmussen et al., 2025; Graphiti contributors, 2026). NuggetIndex combines source spans, validity intervals, and lifecycle states (Zerhoudi et al., 2026). DeFuzzRAG and IA-RAG handle fuzzy dates and interval-based retrieval (Chen et al., 2026; Wang et al., 2026). These mechanisms make date interpretation central to evidence selection. We study the exact decisions available before choosing one interpretation of an uncertain occurrence date.

Our central question is simple: *Which claims can be removed without discarding support under an allowed timeline?* We also ask which claims have support under every allowed timeline. These questions produce two useful sets. *Possible support* retains evidence that could apply somewhere in the requested period. *Guaranteed support* retains evidence that applies somewhere in that period under every permitted date assignment. Claims with possible but unguaranteed support have unresolved temporal status.

The second set requires care. Suppose a claim begins sometime between times 0 and 10. A conflicting claim begins at time 5. If the first claim begins earlier, it contributes before the replacement. If it begins later, it contributes from its own start. Every allowed timeline therefore supports it somewhere in  $[0, 10]$ . Yet there is no single time when all timelines support it. Intersecting the query with a certainly active interval would miss this evidence. Figure 1 illustrates the distinction.

We derive a compact solution for a fixed direct replacement policy. Each claim has an earliest and latest possible start. One witness determines when replacement becomes unavoidable. A second wit-

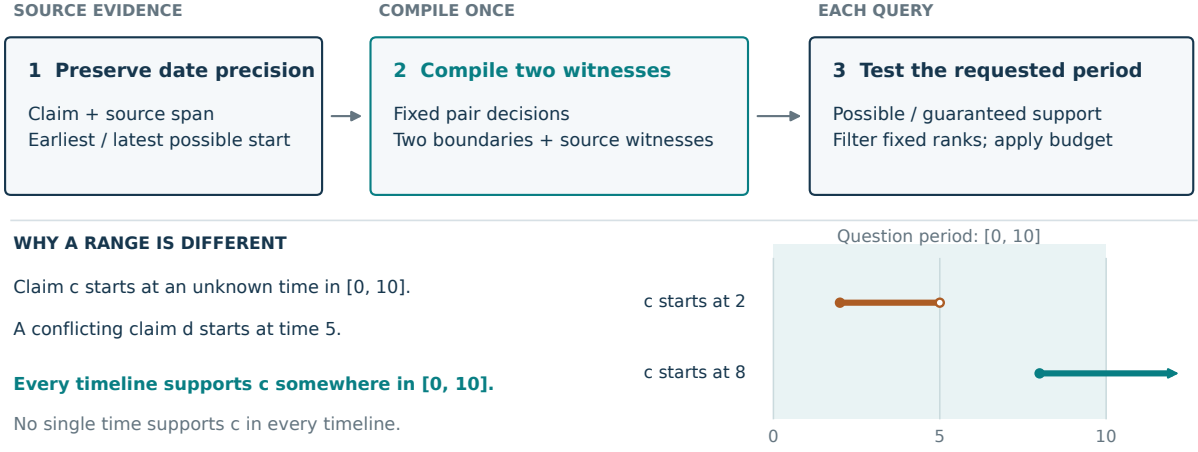


Figure 1: The compiler preserves uncertain occurrence dates and records two direct replacement witnesses per claim. Query-time tests return possible and guaranteed support for the requested period. The lower panel illustrates two allowed timelines. Claim *c* has support somewhere in [0, 10] in every timeline, although no single time guarantees support. The times are illustrative units.

ness determines when replacement becomes possible. These two witnesses answer every point and period query without enumerating timelines. They expose which source decisions caused an exclusion.

Our contribution has three parts. First, we give exact support tests and prove that two witnesses suffice. Two are necessary in the worst case among direct witness subsets. Second, we prove refinement and error-locality properties. We also certify exactly when every timeline returns the same ranked claim context. More precise dates narrow possible support and expand guaranteed support. Changing one directed replacement decision affects only its target claim. Third, we establish a source-grounded evaluation protocol that preserves complete answer sets and original passage text. It separates executor correctness, yearly snapshot selection, and natural-passage retrieval.

This protocol reveals why temporal metrics need the full requested scope. In TempLAMA, a changed first answer often remains valid in the new annual answer set. In TimeQA, a range can include successive answers, concurrent answers, or several names for one answer. The compiler addresses uncertainty in occurrence dates. The evaluation measures how temporal selection interacts with the evidence that a question requests.

## 2 From Sources to Date Constraints

Temporal evidence contains several kinds of dates. The retrieval interface must preserve their meaning before applying any replacement rule. We distin-

| Source meaning                         | Representation and operation                                                            |
|----------------------------------------|-----------------------------------------------------------------------------------------|
| Update occurred in a year              | Bound its unknown start by that year's dates; use the two-witness compiler.             |
| Fact held during a stated period       | Preserve its validity interval; test overlap with the question's period.                |
| Fact was observed in a yearly snapshot | Preserve the observation and all listed answers; do not infer an exact transition date. |

Table 1: Three date meanings require different operations. The compiler takes bounds on one occurrence date.

guish an uncertain occurrence from a known duration and from a dated observation. Table 1 gives the operational difference.

**Occurrence precision.** An exact day gives a point on the chosen time axis. A stated month gives bounds on an occurrence within that month. A stated year gives wider bounds. The representation keeps this precision instead of substituting January 1 or the interval midpoint. Relative expressions require a source anchor. An expression such as *around 1713* does not itself supply a numerical error radius. The source adapter must provide justified bounds before invoking the compiler.

**Replacement decisions.** A directed decision states that one claim replaces another if it occurs strictly later. The decision concerns the same subject and attribute. It must distinguish a changed value from an additional valid value. For example, the Chicago Stadium passage gives overlapping periods for the Blackhawks and the Bulls. Neither

tenancy replaces the other. The compiler takes pair decisions as inputs, together with source identifiers and date bounds. This interface allows the same executor to serve rule-based and learned extractors.

**Question scope.** We study questions asking which claims apply at some time within a specified period. Point questions are the special case with identical start and end dates. A question asking what held throughout a period requires a different predicate. The query adapter preserves this distinction. It preserves the supplied date precision. Thus a question about a whole year does not silently become a question about its last day.

**Source units.** Replacement operates on claims, rather than entire paragraphs. A single paragraph can contain several historical states and several relations. The News Quiz passage, for example, lists successive hosts and a returning host. Removing that paragraph because one host was replaced would remove other requested evidence. Each extracted claim retains its passage identifier and source span. The retrieval layer can recover the supporting text after temporal selection.

**Three output states.** For a given question, the compiler marks each claim as guaranteed, unresolved, or unsupported. A guaranteed claim has support under every permitted timeline. An unresolved claim has support under some, but not all, timelines. An unsupported claim has support under none. The interface exposes these states before the context budget is applied. This preserves the difference between lacking temporal support and merely having uncertain dates.

### 3 Dates as Constraints

A year names a range of dates. It does not identify one update date. Our compiler keeps that range and answers questions with matching temporal scope. It returns two evidence sets: possible support and guaranteed support.

**Occurrence dates and replacement.** Each claim  $c$  has text and an unknown occurrence date  $x_c$ . Its source bounds this date by two inclusive dates,  $\ell_c \leq x_c \leq u_c$ . Dates use a common real-valued time axis. These bounds describe uncertainty about an occurrence, rather than its duration. The allowed assignments form the product  $\Omega = \prod_c [\ell_c, u_c]$ . A fixed text gate  $G(d, c)$  accepts a direct replacement when  $d$  occurs strictly after  $c$ . The gate need not

be transitive. It excludes self-pairs:  $G(c, c) = 0$ . It must distinguish replacement from coexistence. The compiler preserves its decisions across date assignments.

For assignment  $x$ , claim  $c$  is active at date  $t$  when

$$A_x(c, t) = 1[x_c \leq t \wedge \nexists d : G(d, c), x_c < x_d \leq t]. \quad (1)$$

Equal dates do not establish replacement order. A question specifies an inclusive date window  $Q = [a, b]$ . Point questions set  $a = b$ . A claim supports the window when it is active somewhere inside it:

$$S_x(c, Q) = 1[\exists t \in Q : A_x(c, t) = 1]. \quad (2)$$

Possible support requires one allowed assignment with  $S_x(c, Q) = 1$ . Guaranteed support requires this condition in every allowed assignment. Write  $P_Q(c)$  and  $H_Q(c)$  for these two Boolean support decisions.

**Two direct witnesses.** One witness can force retirement despite date uncertainty. Another can permit retirement under a particular assignment. Their boundaries are

$$p_c = \min\{u_d : G(d, c), \ell_d > u_c\}, \quad (3)$$

$$r_c = \min\{\ell_d : G(d, c), u_d > \ell_c\}. \quad (4)$$

An empty minimum is  $+\infty$ . We store one witness for each finite boundary. These compiled certificates require a complete candidate scan. The compiler evaluates candidate gates and updates these two minima. It uses at most two witness records per claim. Two records are necessary in the worst case among direct witness subsets.

**Theorem 1** (Exact window support). *For every claim and inclusive query window,*

$$P_Q(c) \iff \ell_c \leq b \wedge a < p_c, \quad (5)$$

$$H_Q(c) \iff u_c \leq b \wedge (a \leq \ell_c \vee a < r_c). \quad (6)$$

The first condition gives the smallest filter that never removes potentially supported evidence. The second gives the largest filter whose retained claims always have support. Each decision takes constant time after compilation. Appendix A gives the proofs.

Figure 1 shows why guaranteed window support needs its own test. A different witness time can establish support in each permitted timeline. The order of the temporal and assignment quantifiers therefore matters.

**Input:** claims and accepted pairs  $(d, c)$ .  
**Output:**  $(p_c, r_c)$  and their source witnesses.

1. Set  $p_c = r_c = \infty$  for every claim.
2. For each accepted pair with  $d \neq c$ :
3. If  $\ell_d > u_c$ , minimize  $p_c$  using  $u_d$ .
4. If  $u_d > \ell_c$ , minimize  $r_c$  using  $\ell_d$ .
5. Store the supplying witness for each minimum.
6. Break boundary ties by source identifier.

Steps 3–4 execute inside the pair scan.

Figure 2: Two-boundary compilation. Every accepted pair is examined once. The stored minima answer every query window.

**Corollary 1** (Refinement and local influence). *Narrowing date bounds can only shrink possible support and expand guaranteed support. If two gates differ on  $h$  directed pairs, either support mask changes for at most  $h$  claims. With fixed rankings, their filtered top- $k$  contexts differ in at most  $2 \min(k, h)$  items.*

Each directed pair affects only its target claim’s certificates. When all date bounds are exact, both filters recover earliest direct replacement. The formulas therefore extend the same policy without choosing arbitrary dates.

## 4 Compilation and Retrieval

### 4.1 A shared scan for both certificates

The compiler receives claims, date bounds, and accepted directed replacement pairs. It initializes both boundaries to infinity. Each accepted pair updates at most two minima. Figure 2 gives the complete boundary update. The same witness can supply both boundaries. If several witnesses tie, their identifiers determine a stable choice.

Let  $n$  count claims and  $m$  count supplied accepted pairs. Boundary compilation takes  $O(n + m)$  time and stores  $O(n)$  records. A decision for one claim takes  $O(1)$  time. A complete support mask takes  $O(n)$  time. The reference gate interface can discover pairs through a complete directed scan. This scan makes  $n(n - 1)$  gate calls. The certificate representation does not require storing those calls or their rejected pairs.

Candidate discovery and boundary compilation serve different purposes. An inverted index can reduce gate calls when its postings cover every

accepted pair. Incomplete blocking can omit a replacement and change a boundary. Our implementation provides a complete scan as its executable specification. The pair-stream interface supports an existing extractor or a complete index. Both interfaces return the same certificate format.

### 4.2 Filtering a fixed ranking

Let  $\pi_q$  be a complete relevance ranking for question  $q$ . Let  $M(q)$  contain claims with the chosen support status. Retrieval returns

$$R(q) = \text{Top}_k \langle c \in \pi_q : c \in M(q) \rangle. \quad (7)$$

The filter preserves the relative order of retained claims. It runs before the context budget. Possible-support retrieval uses the union of guaranteed and unresolved claims. Guaranteed-support retrieval uses only guaranteed claims. The compiler supplies both masks without changing relevance scores.

These policies serve different evidence needs. Possible support is the smallest pool that preserves every admissible supporting claim. Its complement identifies evidence that can be excluded across all permitted timelines. Guaranteed support contains claims whose temporal support survives every permitted date assignment. Unresolved claims retain their source text and bounds for downstream reasoning. Neither policy turns temporal support into a relevance score.

### 4.3 Certifying an unchanged context

A reader uses a ranked context with a fixed budget  $k$ . The complete ranking fixes all score ties. Let  $P(Q)$  and  $H(Q)$  denote possible and guaranteed support. Write  $\text{Top}_k(S)$  for the first  $k$  ranked claims in  $S$ .

**Theorem 2** (Exact context stability). *Every allowed date assignment returns the same context exactly when*

$$\text{Top}_k(P(Q)) = \text{Top}_k(H(Q)). \quad (8)$$

*Equivalently, every claim in  $\text{Top}_k(P(Q))$  belongs to  $H(Q)$ .*

*Proof.* Let  $C = \text{Top}_k(P(Q))$ . If  $C \subseteq H(Q)$ , every world retains  $C$  and selects the same ranked context. Conversely, take  $c \in C \setminus H(Q)$ . Some world supports  $c$ , while another does not. At most  $k - 1$  possible claims outrank  $c$ . Thus every world supporting  $c$  must select it. The two worlds return different contexts. For  $k = 0$ , all contexts are empty.  $\square$

| Query $Q$ | Possible | Guaranteed | Status      |
|-----------|----------|------------|-------------|
| $[0, 0]$  | Yes      | No         | Unresolved  |
| $[0, 2]$  | Yes      | Yes        | Guaranteed  |
| $[1, 2]$  | Yes      | No         | Unresolved  |
| $[2, 4]$  | Yes      | No         | Unresolved  |
| $[5, 5]$  | No       | No         | Unsupported |

Table 2: Illustrative decisions for  $c = [0, 2]$ ,  $p_c = 5$ , and  $r_c = 1$ . A larger query window can have guaranteed support even when its final point does not.

This test identifies uncertainty that affects the returned context. Lower-ranked claims can remain uncertain without changing retrieval. The test needs two filtered rankings and no enumeration of date assignments. Its guarantee concerns retrieved evidence, rather than generated answers.

#### 4.4 A worked certificate

Consider three claims with occurrence bounds  $c = [0, 2]$ ,  $d = [1, 4]$ , and  $e = [3, 5]$ . Accept the directed replacements  $d \rightarrow c$  and  $e \rightarrow c$ . Claim  $d$  can occur after  $c$ , but it need not do so. Its earliest allowed start supplies  $r_c = 1$ . Claim  $e$  must occur after  $c$ . By time 5 it has occurred under every assignment, so it supplies  $p_c = 5$ . Table 2 shows the resulting decisions.

The two witnesses preserve different decisions. If we retain only  $d$ , time 5 incorrectly becomes possible for  $c$ . If we retain only  $e$ , time 2 incorrectly becomes certain. Thus neither one-witness subset reproduces all outputs. This construction establishes the worst-case two-witness requirement. The lower bound concerns retained source witnesses, rather than an unrestricted numerical encoding.

The example also separates date ambiguity from retrieval ambiguity. For  $Q = [0, 2]$ , rank  $c$  before  $d$  and set  $k = 1$ . Claim  $c$  is guaranteed, so every timeline returns the same context. Claim  $d$  remains unresolved below the budget. Increasing the budget to two makes the context depend on whether  $d$  has started. The stability test detects that change without enumerating timelines.

Now suppose later evidence narrows the same dates to  $x_c = 2$ ,  $x_d = 1$ , and  $x_e = 5$ . Every refined date remains within its original bounds. Claim  $d$  can no longer replace  $c$ , since it occurs earlier. Both certificates now identify  $e$ , and  $c$  is active on  $[2, 5]$ . The window  $[2, 4]$  becomes guaranteed. The point  $[0, 0]$  becomes unsupported. The refinement removes possible support and resolves uncertainty into guaranteed support, as predicted by the corol-

lary.

#### 4.5 Why the query tests are exact

The possible-support boundary follows from a worst-case ordering test. A witness whose earliest start exceeds  $c$ 's latest start must follow  $c$ . Once that witness's latest date passes, it has retired  $c$  in every timeline. Before the first such boundary, each accepted witness can occur before  $c$  or after the queried time. Independent occurrence bounds permit these choices together. This gives the possible point set  $[\ell_c, p_c]$ .

For guaranteed window support, first ensure that  $c$  starts by the window's end in every timeline. This is the condition  $u_c \leq b$ . Any start inside the window immediately supplies support. Failure therefore requires an earlier start followed by replacement before the window opens. A witness can produce that failure exactly when  $a > \ell_c$  and  $r_c \leq a$ . Negating this condition gives the guaranteed-support test.

This argument explains why simply intersecting two intervals is insufficient. Possible support has an ordinary pointwise interval. Guaranteed window support instead depends on whether an entire timeline can avoid support throughout the query. The compiler answers that question directly.

#### 4.6 Stable updates and inspectable decisions

A certificate contains the target identifier, its date bounds, both minima, and their witness identifiers. A retirement-based exclusion traces to its replacement witness. A date-based exclusion traces to the target bounds. Changing that decision updates only the target's two minima. This is the dependency underlying Corollary 1. Transitive grouping creates a different dependency: one edge can merge entire histories. Direct compilation avoids that propagation.

The error bound concerns eligibility and fixed-ranked contexts. For  $h$  changed directed pairs, at most  $h$  target masks can change. The selected sets differ by at most  $2 \min(k, h)$  items. A removal can cause one new item to enter the context. It can shift several positions within the returned list. The bound concerns selected sets, rather than ordered positions.

Fix the existing date bounds and replacement decisions. Adding claims whose lower date bounds exceed the query end preserves earlier support decisions. Such a claim cannot start within that window or retire another claim by its end. This follows

from the same direct support definition. If an added source instead reveals an earlier event, the historical mask can change. The date bound determines the effect, rather than the ingestion order.

Finally, exact dates recover a familiar special case. Possible and guaranteed support coincide. One earliest direct replacement determines each claim’s expiry. The uncertain-date compiler extends this policy while preserving all permitted occurrence orders.

## 5 Evaluation Protocol

We evaluate compiler correctness and the evidence conditions that temporal retrieval must handle. The corpus experiments compare temporal baselines on annual records and natural paragraphs. An independent enumeration tests the new uncertainty compiler against its possible-world definition.

**Complete annual answer sets.** We use all 34,963 original TempLAMA test records across 4,057 subject–relation keys (Dhingra et al., 2022). Rendering every listed answer produces 45,866 positive observations. Each observation contains its source sentence, an opaque identifier, and its answer year. This year specifies the benchmark target time, rather than document publication or arrival. We evaluate only years with an original answer set. Multiple annual answers can describe concurrent facts or succession within one year. A missing answer creates no retirement event.

We freeze a key-disjoint split before evaluating the new conditions. The 300 keys used in earlier development remain excluded from test. A fixed hash assigns 764 additional keys to development and 2,993 keys to test. The resulting test contains 25,752 annual questions. All methods search the same complete pool. The nine public question templates define text-only subject, relation, and value parsers. Source entity identifiers and answer identifiers enter scoring only. Six textual slots merge distinct source entities with identical question wording. We retain these cases and publish their identifiers.

**Natural paragraphs.** TimeQA supplies 33,493 original paragraphs from 749 Wikipedia articles (Chen et al., 2021). The index includes each visible article title. Our primary track contains 830 human questions with nonempty annotated answer spans. A diagnostic track uses 2,613 questions generated from official templates over the same

articles. These tracks exclude 159 and 384 empty-span placeholders, respectively. We identify these placeholders before inspecting retrieval outcomes. Every retained span matches its original paragraph substring exactly. Human questions form a query variant of this corpus.

**Temporal baselines.** We use BM25 with  $k_1 = 1.5$  and  $b = 0.75$  (Robertson et al., 1995). The common context budget is five observations. The annual experiment includes recent-date tie breaking, date-first ranking, and exponential age weighting. Development selects the age half-life from  $\{0.5, 1, 2, 5, 10, \infty\}$  years using mean key-level Hit@1. The selected half-life is two years. The original lexical predicate closes an observation at its earliest matched different-value observation. A matched Graphiti condition executes verified upstream temporal code with text-parsed slots and values (Graphiti contributors, 2026). This condition evaluates the temporal policy kernel. Query latest-value returns one observation from the latest matching year. Query latest-batch retains every observation tied at that year.

Natural-paragraph policies extract year mentions and explicit year ranges from text. Latest-year reranks twenty BM25 candidates by the latest mentioned year not after the question’s upper bound. Year-envelope filtering retains paragraphs whose mentioned-year span overlaps the requested period. Range filtering requires overlap with an explicit range when one is present. Range-support reranking promotes overlapping ranges within twenty candidates. Both filters retain paragraphs without their required temporal feature. We freeze these rules before execution.

**Measurements.** Annual Hit@1 requires the correct source key and an accepted answer identifier. Answer Recall@5 measures the fraction of distinct accepted answers represented in five observations. Stale@5 records a retrieved target-key value absent from that year’s answer set. Candidate suppression measures accepted values removed before the final context selection. TimeQA Hit@5 measures whether five retrieved paragraphs contain an annotated answer span. Span Recall@5 measures the fraction of annotated spans covered. These metrics assess evidence selection. Confidence intervals resample complete keys for TempLAMA and complete articles for TimeQA. We use 5,000 and 2,000 resamples, respectively, with seed 20261006.

| Source-label property | Count        | Rate (95% CI)     |
|-----------------------|--------------|-------------------|
| Multiple answers      | 6,498/25,752 | 25.2 [24.2, 26.2] |
| Old first retained    | 1,456/3,980  | 36.6 [35.1, 38.0] |
| Any answer retained   | 6,050/6,961  | 86.9 [86.0, 87.8] |

Table 3: Answer-set properties on held-out source keys. The last two rows condition on first-answer changes and complete-set changes, respectively. Rates and intervals are percentages.

| Policy             | H@1 ↑ | R@5 ↑ | S@5 ↓ |
|--------------------|-------|-------|-------|
| BM25               | 64.64 | 80.84 | 45.89 |
| BM25 + recent ties | 84.04 | 90.10 | 40.50 |
| Age weighting      | 99.81 | 99.98 | 24.07 |
| Date-first ranking | 99.81 | 99.98 | 0.00  |
| Original predicate | 99.37 | 99.77 | 0.77  |
| Graphiti kernel    | 99.78 | 99.93 | 0.00  |
| Query latest-value | 99.89 | 86.37 | 0.00  |
| Query latest-batch | 99.89 | 99.98 | 0.00  |

Table 4: Annual source-label retrieval on 25,752 held-out questions. All values are percentages. H@1: first-result accuracy; R@5: answer recall; S@5: target-label stale exposure.

## 6 Results

### 6.1 Changing answers do not define replacement events

Table 3 characterizes the untouched TempLAMA test keys. A changed first-listed answer often leaves the previous answer accepted in the same annual label set. Most changes to the complete answer set also retain at least one answer. These observations establish the need to preserve alternatives before selecting a retirement rule. They also separate annual answer membership from event timing. The released audit includes every transition, its complete answer sets, and its source identifier.

### 6.2 Strong date policies recover annual answer sets

Table 4 compares the fixed baseline policies on all held-out annual questions. Date-first ranking nearly saturates this source-derived task. Query latest-batch also preserves multiple accepted answers within the returned context. Returning only one latest value instead suppresses 13.63% of accepted answers before context selection. The key-cluster confidence interval is [13.06, 14.21] percentage points. This loss concerns answer coverage even when the first returned answer is correct.

The age policy shows a different effect. Its first-result accuracy remains high while older unsupported values still enter the five-observation con-

| Policy         | Human |       | Templates |       |
|----------------|-------|-------|-----------|-------|
|                | H@5   | R@5   | H@5       | R@5   |
| BM25           | 45.66 | 45.04 | 32.49     | 31.75 |
| Latest year    | 54.70 | 54.02 | 51.86     | 50.89 |
| Year envelope  | 49.16 | 48.33 | 34.44     | 33.67 |
| Explicit range | 44.10 | 43.53 | 32.11     | 31.39 |
| Range support  | 44.58 | 44.00 | 38.50     | 37.63 |

Table 5: TimeQA paragraph retrieval percentages. H@5 measures annotated-passages coverage; R@5 measures annotated-span recall. The tracks contain 830 human questions and 2,613 template questions.

text. Thus first-result accuracy does not measure the temporal composition of a retrieval context.

The Graphiti policy and exact direct-first compilation produce identical endpoints on all 45,866 observations. Their equivalence uses identical text-parsed candidate decisions. The original lexical predicate uses a different matcher and remains a separate condition. These comparisons establish the deterministic reference behavior for the uncertainty compiler. We assess uncertain event times through the independent verification below.

### 6.3 Natural paragraphs reward ranking and expose deletion costs

Table 5 reports retrieval from the complete TimeQA paragraph pool. Latest-year reranking improves human-question Hit@5 by 9.04 percentage points over BM25. Its article-cluster confidence interval is [5.54, 12.69]. The template diagnostic gains 19.36 points, with interval [16.95, 21.78]. The comparison provides a strong lexical baseline for subsequent temporal evidence selection.

Hard filtering can remove annotated support even when its aggregate retrieval score improves. Year-envelope filtering removes 284 of 2,840 template-track spans and 107 of 887 human-track spans. Explicit-range filtering removes 170 and 63 spans, respectively. Paragraphs can describe several events or omit a relevant date from the answer sentence. A paragraph’s mentioned-year range therefore needs a separate semantic check before it justifies evidence removal. The compiler isolates this evidence decision from its temporal guarantees.

### 6.4 Independent verification of the uncertainty compiler

An independent implementation enumerates possible worlds and compares their memberships with the compiled bounds. Across 253,050 predicate

checks, the implementation finds no disagreement. The checks include equal times, uncertain order, and interval-valued event dates. These are exhaustive correctness checks on finite instances, rather than corpus accuracy measurements. A separate oracle checks context invariance across 65,809 nonempty mask families. All 328,747 budget-specific decisions agree with the theorem. Interface tests also compare 6,912 contexts from finite event-date assignments. The release contains the enumeration code, test logs, and complete corpus predictions.

## 7 Related Work

**Temporal memory and replacement.** Graphiti closes earlier facts through direct contradictory evidence (Rasmussen et al., 2025; Graphiti contributors, 2026). MemStrata applies deterministic supersession to structured keys (Yadav, 2026). ERASE edits stored fact histories as new documents arrive (Li et al., 2025). NuggetIndex attaches source spans, validity intervals, and lifecycle states to atomic records (Zerhoubi et al., 2026). It distinguishes functional and multi-valued relations. These systems establish direct invalidation and cardinality-aware memory as existing mechanisms. Our contribution concerns exact compilation across uncertain occurrence dates under fixed direct decisions.

**Temporal retrieval.** Re<sup>3</sup> combines fact grouping, conflict filtering, and temporal relevance (Cao et al., 2026). DeFuzzRAG adapts temporal granularity and filters inferred document scopes by query overlap (Chen et al., 2026). IA-RAG represents intervals and refines uncertain event dates (Wang et al., 2026). TA-RAG retrieves evidence across timelines (Lau et al., 2025). These methods motivate interval-aware evidence selection. We preserve all assignments within supplied occurrence bounds. The resulting compiler distinguishes possible support from guaranteed support within a window. The latter uses a different quantifier order from overlap with certainly active points.

**Uncertain temporal data.** Possible and certain temporal semantics have a substantial database foundation. Anselma et al. (2013) study valid-time indeterminacy and compact representations. Fan et al. (2012) study data currency without reliable timestamps. Amarilli et al. (2017) analyze possible and certain answers over order-incomplete

data. Our model restricts uncertainty to independent occurrence bounds with a fixed replacement predicate. Within this model, two direct witnesses answer all support queries. The witness bound and local influence result concern this specific compilation problem.

**Temporal evaluation.** TimeQA supplies natural passages and questions about dated histories (Chen et al., 2021). TempLAMA evaluates factual answers at yearly resolution (Dhingra et al., 2022). StreamingQA and FreshLLMs study changing information access (Liška et al., 2022; Vu et al., 2024). FRESCO distinguishes obsolete evidence from current but insufficient passages (An et al., 2026). Our protocol preserves complete answer arrays and original answer spans. It separates snapshot observations from occurrence bounds. This distinction prevents a benchmark’s date labels from silently becoming stronger temporal assumptions.

## 8 Conclusion

Temporal retrieval should preserve the precision of both sources and questions. We derive two compiled witnesses that determine possible and guaranteed support across all allowed occurrence dates. The result handles period questions directly, including cases with no certainly active point. Its exact tests, refinement behavior, and local dependencies provide a compact temporal selection interface. Complete-answer and natural-passage evaluations establish the accompanying evidence protocol.

## Limitations

The exactness results assume independent occurrence-date bounds and fixed directed replacement decisions. Correlated dates can make the product of bounds include impossible timelines. The resulting masks remain conservative, but need not be sharp. Incorrect extraction can supply incorrect bounds or replacement decisions. The compiler guarantees execution of those inputs, rather than their semantic correctness. It handles questions asking for support somewhere within a period. Universal coverage throughout a period requires a different query predicate.

The independent oracle validates the compiler on exhaustive finite configurations and selected fractional endpoint cases. The mathematical proofs establish the general result. The corpus experiments measure full-answer snapshot selection and natural-passage retrieval. They do not yet measure the new uncertainty compiler inside a learned claim-extraction and answer-generation pipeline. The TimeQA baselines are source-only executor diagnostics. They are not reproductions of complete DeFuzzRAG, IA-RAG, or NuggetIndex systems.

TempLAMA is a structured diagnostic benchmark with template-rendered claims. Its annual answer sets can include successive values within one year. TimeQA supplies natural passages, but some answer anchors are aliases or mentions outside the relevant event description. Annotated-span retrieval differs from temporal entailment and generated-answer correctness. The human questions overlap the larger TimeQA test collection. Our source audit removes empty positive labels and separates development articles from test articles.

Archived reader outputs use the original selected-answer task. No new reader run supports the claims in this revision. The older RealProse excerpts contain source-support problems documented in the appendix. The release keeps those results as historical records. They do not serve as primary natural-language evidence for the new compiler.

## References

Antoine Amarilli, Mouhamadou Lamine Ba, Daniel Deutch, and Pierre Senellart. 2017. [Possible and certain answers for queries over order-incomplete data](#). In *24th International Symposium on Temporal Representation and Reasoning (TIME 2017)*, volume 90

of *Leibniz International Proceedings in Informatics*, pages 4:1–4:19.

Sohyun An, Hayeon Lee, Shuibenyang Yuan, Chun-cheng Jason Chen, Cho-Jui Hsieh, Vijai Mohan, and Alexander Min. 2026. FRESKO: Benchmarking and optimizing re-rankers for evolving semantic conflict in retrieval-augmented generation. *arXiv preprint arXiv:2604.14227*.

Luca Anselma, Paolo Terenziani, and Richard T. Snodgrass. 2013. [Valid-time indeterminacy in temporal relational databases: Semantics and representations](#). *IEEE Transactions on Knowledge and Data Engineering*, 25(12):2880–2894.

Jiawei Cao, Jie Ouyang, Mingyue Cheng, Zhaomeng Zhou, Chunli Liu, Yupeng Li, Zirui Liu, and Shijin Wang. 2026. [Re<sup>3</sup>: Relevance & recency retrieval for mitigating temporal hallucination](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25735–25760. Association for Computational Linguistics.

Ling-Chun Chen, Hsi-Wen Chen, and Ming-Syan Chen. 2026. [DeFuzzRAG: Handling fuzzy time expressions for temporal robustness in retrieval-augmented generation](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30252–30260.

Wenhu Chen, Xinyi Wang, and William Yang Wang. 2021. A dataset for answering time-sensitive questions. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.

Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. 2022. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics*, 10:257–273.

Wenfei Fan, Floris Geerts, and Jef Wijsen. 2012. [Determining the currency of data](#). *ACM Transactions on Database Systems*, 37(4).

Graphiti contributors. 2026. Graphiti: Edge maintenance implementation. [https://github.com/getzep/graphiti/blob/main/graphiti\\_core/utils/maintenance/edge\\_operations.py](https://github.com/getzep/graphiti/blob/main/graphiti_core/utils/maintenance/edge_operations.py). Source snapshot accessed October 6, 2026.

Kwun Hang Lau, Ruiyuan Zhang, Weijie Shi, Xiaofang Zhou, and Xiaojun Cheng. 2025. Reading between the timelines: RAG for answering diachronic questions. *arXiv preprint arXiv:2507.22917*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Belinda Z. Li, Emmy Liu, Alexis Ross, Abbas Zeitoun, Graham Neubig, and Jacob Andreas. 2025. [Language modeling with editable external knowledge](#). In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3070–3090. Association for Computational Linguistics.

Adam Liška, Tomáš Kočíský, Elena Gribovskaya, Tayfun Terzi, Eren Sezener, Devang Agrawal, Cyprien de Masson d’Autume, Tim Scholtes, Manzil Zaheer, Susannah Young, Ellen Gilsonan-McMahon, Sophia Austin, Phil Blunsom, and Angeliki Lazaridou. 2022. StreamingQA: A benchmark for adaptation to new knowledge over time in question answering models. In *International Conference on Machine Learning (ICML)*.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. [Zep: A temporal knowledge graph architecture for agent memory](#). *arXiv preprint arXiv:2501.13956*.

Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, and Mike Gatford. 1995. Okapi at TREC-3. *NIST Special Publication*, 500-225:109–126.

Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. 2024. FreshLLMs: Refreshing large language models with search engine augmentation. In *Findings of the Association for Computational Linguistics (ACL)*.

Xiaoman Wang, Yaoze Zhang, Wenzhuo Fan, Hongwei Zhang, Ding Wang, Guohang Yan, Song Mao, Botian Shi, Yunshi Lan, and Pinlong Cai. 2026. IA-RAG: Interval-algebra-driven temporal reasoning for dynamic knowledge retrieval. *arXiv preprint arXiv:2606.06044*.

Neeraj Yadav. 2026. [Temporal validity in retrieval memory: Eliminating stale-fact errors for AI agents over evolving knowledge](#). *arXiv preprint arXiv:2606.26511*.

Saber Zerhouni, Michael Granitzer, and Jelena Mitrović. 2026. [NuggetIndex: Governed atomic retrieval for maintainable RAG](#). In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

## A Proofs for Uncertain Dates

All date assignments belong to the product  $\Omega$  defined in Section 3. The replacement gate remains fixed across these assignments. The proofs use real dates with closed input bounds. Every witness differs from its target:  $G(c, c) = 0$ .

### A.1 Point support

**Lemma 1** (Exact point support). *For every date  $t$ ,*

$$P_{[t,t]}(c) \iff \ell_c \leq t < p_c, \quad (9)$$

$$H_{[t,t]}(c) \iff u_c \leq t \wedge (t \leq \ell_c \vee t < r_c). \quad (10)$$

*Proof.* If  $t < \ell_c$ , no assignment makes  $c$  active. If  $t \geq p_c$ , a witness satisfies  $\ell_d > u_c$  and  $u_d \leq t$ . It occurs strictly after  $c$  and by  $t$  in every assignment. Thus possible support requires  $\ell_c \leq t < p_c$ .

Conversely, assume these inequalities. Choose  $x_c = \min(u_c, t)$ . For each accepted witness, choose  $x_d \leq x_c$  when its bounds permit that choice. Otherwise,  $\ell_d > x_c$ . If  $x_c = t$ , this witness occurs after  $t$ . If  $x_c = u_c < t$ , the definition of  $p_c$  gives  $u_d > t$ . Choose the witness after  $t$  in this case. Independent date bounds permit all these choices simultaneously. No witness then retires  $c$  by  $t$ .

For certain support, every assignment has  $x_c \leq t$  exactly when  $u_c \leq t$ . Assume this condition. A witness can retire  $c$  by  $t$  exactly when some assignment satisfies  $x_c < x_d \leq t$ . Choose  $x_c = \ell_c$ . A suitable witness date exists exactly when  $t > \ell_c$ ,  $\ell_d \leq t$ , and  $u_d > \ell_c$ . The minimum eligible  $\ell_d$  is  $r_c$ . Possible retirement therefore requires  $t > \ell_c$  and  $r_c \leq t$ . Its complement gives the stated condition.  $\square$

The equality term is necessary. Take  $[\ell_c, u_c] = [0, 0]$  and  $[\ell_d, u_d] = [0, 1]$  with  $G(d, c)$  true. Claim  $c$  is certainly active at zero. A strictly later replacement cannot have occurred by zero. It is not certainly active at any later date. An ordinary half-open interval cannot represent this singleton. The implementation retains the exact comparisons from the lemma.

### A.2 Window support

*Proof of Theorem 1.* Possible point support occupies  $[\ell_c, p_c)$  by Lemma 1. This set intersects  $[a, b]$  exactly when  $\ell_c \leq b$  and  $a < p_c$ . The two existential quantifiers commute, proving Equation 5.

For guaranteed support, first suppose  $u_c > b$ . The assignment  $x_c = u_c$  gives no support within the window. Thus  $u_c \leq b$  is necessary. Now assume that inequality. Every assignment with  $x_c \geq a$  supplies support at  $x_c$  itself. For an earlier occurrence, support fails exactly when a direct witness retires  $c$  by  $a$ . Such an assignment exists exactly when some witness satisfies  $x_c < x_d \leq a$ . The preceding lemma shows this is equivalent to

$a > \ell_c$  and  $r_c \leq a$ . Taking its complement proves Equation 6.  $\square$

For each world, support means existence of a suitable time. The suitable time can differ across worlds. Consequently, a window can have guaranteed support even when no point has certain support. This distinction remains after exact text matching.

### A.3 Sharpness and refinement

The possible filter retains precisely the union of world-specific support sets. Any smaller filter omits a supported claim in some world. The guaranteed filter retains precisely their intersection. Any larger filter includes an unsupported claim in some world. These statements establish the claimed minimal and maximal filters. They concern each retained claim’s support, rather than one jointly optimal context.

Narrowing date bounds replaces  $\Omega$  with a subset  $\Omega' \subseteq \Omega$ . Existential support can only disappear when its assignment set shrinks. Universal support can only appear. This proves refinement monotonicity without assumptions about gate transitivity. For correlated dates, a product approximation instead gives conservative bounds. It can enlarge possible support and reduce guaranteed support.

### A.4 Local gate influence

Suppose gates  $G$  and  $H$  differ on directed pairs with target set  $U$ . Every claim outside  $U$  uses identical terms in both boundary minima. Its support status therefore remains unchanged for every query window. Thus either mask difference lies within  $U$ , where  $|U| \leq h$ .

Fix a complete candidate ranking and deterministic tie handling. Changing one eligibility bit alters at most one selected item and one replacement. Apply this argument successively to the targets in  $U$ . The resulting context difference is at most  $2|U|$ . Each context also contains at most  $k$  items. The difference is therefore at most  $2 \min(k, |U|) \leq 2 \min(k, h)$ .

A directed gate edit differs from a symmetric matching edit. A symmetric edit can change both directions and affect two target claims. Overlapping date bounds do not define one fixed older endpoint. A date edit can also affect several target certificates. The local bound does not treat either edit as one directed decision.

### A.5 Exact dates and compilation cost

When  $\ell_c = u_c = x_c$  for every claim, both boundary minima select the earliest strictly later direct witness. The possible and guaranteed masks then coincide for every window. For a point query, they equal the exact-date earliest-witness policy.

The reference compiler evaluates all  $n(n-1)$  candidate directed gates. The certificates require this scan or a proved complete candidate index. Two stored witnesses alone cannot verify that no omitted witness changes a boundary. If accepted pairs are already available, two minima require  $O(n+m)$  time for  $m$  pairs. The output contains at most  $2n$  witness records. Every point or window decision takes  $O(1)$  time per claim. A full mask takes  $O(n)$  time. A complete text index can reduce gate evaluations without changing these results. No subquadratic guarantee is asserted for candidate discovery.

The worked certificate in Section 4 proves worst-case witness minimality.

## B Source and Evaluation Details

### B.1 TempLAMA source reconstruction

We recover the complete official test release and a public alias mirror. Both contain the same 34,963 records and the same answer identifiers. The normalizer verifies record IDs, query text, first answers, and full answer sets. The source manifest stores byte hashes for both inputs. It preserves all 45,866 answer observations.

A source key combines the subject identifier and relation identifier. The revised evaluation reserves all 300 previously used keys for legacy development. A deterministic key split assigns 764 additional keys to development and 2,993 to test. No key appears in two partitions. The test contains 25,752 yearly records and 33,908 answer observations. All thresholds and baseline choices use development records.

A year is a benchmark observation bucket. It is not a publication timestamp or an independently observed update date. Multiple answers can reflect within-year succession or concurrent values. An answer absent from a later label set is not independently verified as retired. The source audit consequently reports changes in label sets. It does not convert these changes into new occurrence bounds.

Textual names do not always identify one source entity. Six parsed slots merge 13 source entities. Fifty identical question-year groups cover 105

source rows. Sixty-one of those rows occur in the held-out test partition. The evaluation keeps these cases. It does not expose hidden entity identifiers to the text parser.

## B.2 TimeQA provenance and positive labels

The source repository is pinned to commit 38b0598; the manifest records its full identifier. The release includes immutable download URLs, SHA256 hashes, and Git blob checks. The normalizer preserves each original paragraph and its character offsets. Repeated relation histories reuse the same paragraph identifier. The test corpus contains 33,493 distinct paragraphs.

The test annotations contain 384 empty answers with zero-length spans. The human subset contains 159 such annotations. The positive retrieval tracks exclude these placeholders. They retain 2,613 template-rendered questions and 830 original human questions. The human track is a subset of the larger test collection. It is not an independent second benchmark.

Eight articles occur in both the published development and test collections. Removing them before development produces 2,651 nonempty development questions. The natural-passage controls have fixed rules and no fitted parameters. The source audit records all removals and overlapping article paths.

A matched string span identifies the benchmark’s retrieval anchor. Some anchors appear in headings or in passages about another period. Aliases can also create several labels for one entity. The evaluation therefore names its metrics annotated passage hit and annotated span coverage. Entailment requires a separate source-support judgment.

## B.3 Source-grounded examples

The Chicago Stadium paragraph gives two overlapping tenancies. The Blackhawks’ stated period is 1929–1994. The Bulls’ stated period is 1967–1994. Both cover the question’s period, 1967–1988. This example supports coexistence directly from the passage.

The News Quiz paragraph describes several hosts between 1977 and 1995. It includes Barry Norman, Barry Took, and Simon Hoggart. Barry Took returns after an intervening host. This example illustrates range coverage and recurrence. Its multiple answers do not establish simultaneous hosts.

A Ruud Lubbers paragraph mixes month-level and day-level employment dates. The associated question uses year-level bounds. Other passages use expressions such as *before 1516* and *around 1713*. These examples motivate preserving source precision. They do not supply numerical uncertainty widths by themselves. The release records their original text and exact offsets.

## C Archived Results and Source Support

The supplement preserves previous generations, metrics, and source files. These records use a selected-answer protocol. Their answer metric checks normalized target containment. It does not require the complete answer set. No fresh reader inference is reported in this revision.

The previous TempLAMA builder kept the first listed answer. Its 300 keys contain 447 changes in that selected answer. In 153 cases, the previous answer remains in the new full answer set. The revised scorer instead accepts every official answer identifier. Tables in the main body use this complete-source protocol.

The archived extractive reader accesses an annotated value field. The archived historical evaluator also uses a gold anchor. Those outputs remain reproducible as historical records. They are separate from the source-only controls in the main body.

The RealProse source audit identifies 17 excerpts without their assigned subject-role statement. It identifies 33 excerpts without the assigned start year. One excerpt gives a different start year. These findings concern the supplied passages, rather than the underlying facts. TimeQA provides the primary natural-text retrieval source in this revision.

## D Reproduction

The release includes the paper sources, vector figures, compilers, corpus normalizers, predictions, and result cards. A verification entry point reruns the compiler checks and reconstructs the paper tables. Longer entry points rerun full-source and natural-passage retrieval. Input and prediction hashes identify the exact evaluated artifacts. The Graphiti policy adapter includes the inspected upstream code segments and their license. It isolates temporal invalidation under supplied candidates. It does not reproduce Graphiti’s semantic extraction model.